Logistic Regression vs. Gradient Boosting on an Anonymised Tabular Dataset

Matric No.: G2610888C **Name:** Bai Jinfan **IN6227-Assignment-1 Variant-2**

Model: `Claude Opus 5.5 (claude-opus-5-5)` | Interface: Agent harness - Claude Code 2.1.278 (desktop app) | Code: `https://github.com/BreezyB1n/in6227-assignment1`

Summary. We predict the binary `label` (positive class `yes`, 24.0% of training rows) from 15 anonymised features and compare an L2-regularised Logistic Regression (LR) with Histogram Gradient Boosting (HGB). HGB ranks positives better (test AP 0.707 vs. 0.700, 95% CI of the difference +0.001 to +0.014), but at the tuned operating points the two are statistically tied on F1: the signal is largely additive, and threshold choice matters more than model choice.

1 Data exploration and cleaning

The training file has 31,112 rows and the test file 13,334; each has 15 features (7 numeric, 8 categorical) with obfuscated names, so every decision rests on statistics rather than domain knowledge. The profiler (column statistics, single-feature AUCs, linear-redundancy, zero-block and drift detectors) raised 22 flags (Table 1). Train and test distributions match (all KS statistics ≤ 0.009, no unseen levels), so the provided split is used unchanged. Two findings shaped the plan. First, `composite_rank` is an exact weighted sum of two other columns. Second, zeros occur in blocks: the 265-row block is entirely negative, which looks structural (e.g. "nothing recorded"). No single feature exceeds AUC 0.779, so there is no sign of label leakage, but this block could encode post-outcome information. Outliers were never deleted: the test set has the same extremes, so removing them would make CV optimistic.

Table 1: Data issues found by the profiler and the treatment chosen (all statistics learned inside CV folds).

Issue	Evidence	Treatment and rationale
Missing labels	3 train / 1 test rows	Dropped: a label cannot be imputed without inventing ground truth.
Class imbalance	24.0% positive; ratio 3.17:1	Primary metric AP; `class_weight` searched; threshold tuned on out-of-fold predictions; no resampling.
Missing feature values	at most 7 cells per column; 47 train rows affected	Numeric: median + missing indicator; categorical: own `__missing__` level. Fitted per CV fold.
Co-occurring zeros	`performance_score=stability_index=0` in 265 rows (all `no`); `load_ratio=activity_duration=0` in 38	$0 \rightarrow$ NaN + indicator, on prior grounds (isolates the group in one binary feature, removes a point mass). Ablations A1/A2 show no measurable effect.
Heavy tails	1.8% (`load_ratio`) and 3.8% (`activity_duration`) beyond 3 IQR	Rows kept (test has the same extremes). LR only: winsorise at fold-learned 0.5%/99.5% quantiles (A3).
Exact derived feature	`composite_rank` = 0.006*aptitude_score + 0.004*performance_score (R^2=0.9999999)	LR: drop (perfectly collinear, non-identifiable). HGB: keep (trees cannot form linear combinations) (A4, A5).
Rare / placeholder levels	17 `region` levels <1% (4.8% of rows); `species`=Unknown has 8.7% positives	Pool levels below 1% (too few rows for a stable one-hot coefficient); keep Other/Unknown as levels because their positive rates differ from the base rate.
Correlated numerics	r=0.89 (`performance_score`, `stability_index`)	Keep; regularisation copes. Caveat: coefficients and importances are shared.

2 Feature selection and engineering

No column is dropped globally; decisions differ by model because their inductive biases differ. *LR* drops `composite_rank` (it adds no information to a linear model and makes the coefficients non-identifiable), winsorises the three heavy-tailed columns, one-hot encodes categoricals after pooling levels below 1% into one *infrequent* level, and standardises (73 inputs incl. indicators). *HGB* keeps `composite_rank`, because axis-aligned splits cannot synthesise a linear combination; it needs no scaling or clipping (splits depend only on ranks) and uses native categorical splits on the same pooled levels (22 inputs). Both models get the zero$\rightarrow$NaN + indicator treatment and an own level for missing categories. Deliberately *not* done: target encoding (at most 21 levels; pooling suffices), SMOTE (class weighting and threshold tuning handle imbalance without synthetic points), and importance-based feature selection (it would reuse the data that evaluate it).

3 Model training

Why this pair. The comparison asks whether the label is explained by additive effects (LR) or needs interactions and non-linear thresholds (HGB). Rejected alternatives: an RBF-SVM (no native probabilities, $O(n^2)$ training at 31k rows), k-NN (distances over 73 mixed one-hot dimensions are poorly scaled), a single tree (dominated by boosting). **Protocol.** Stratified 5-fold CV on the training set only; randomised search with 20 (LR) and 30 (HGB) candidates, refit on average precision (threshold-free and focused on the minority class, unlike accuracy, which the 76% majority baseline already scores well on), with `class_weight`$\in$ {none, balanced} searched for both (none won). **Stopping criteria:** LR runs L-BFGS to gradient tolerance 10^{-4} (converged after 213 of 5,000 iterations); HGB stops when validation log-loss on an internal 10% split fails to improve for 30 rounds (198 of 2,000 rounds). **Threshold:** the probability cut-off that maximises positive-class F1 on out-of-fold training predictions (0.32 for LR, 0.35 for HGB); the test set is never used for any choice. No selected value lies on a search-range boundary, and the CV train–validation AP gap is small (LR 0.003, HGB 0.037), so over-fitting is limited (larger for HGB, as expected from its capacity).

4 Evaluation and comparison

Both models far exceed the prior (AP 0.238). HGB is ahead on the threshold-free metrics: the test AP difference is +0.007 (bootstrap p=0.022), consistent with CV (+0.011, corrected t-test p=0.031). At the tuned thresholds the classifiers disagree on 220 + 337 test rows and HGB is right on 337 of them (McNemar p <0.001), yet positive-class F1 (CI -0.001 to +0.010) and

Table 2: Hyper-parameter search (randomised, stratified CV, refit on the primary metric) and fitted models. Gap = CV train − validation primary metric (over-fitting indicator).

Model	Search	Selected	Stopping	#inputs	Gap
Logistic Regression	20 cand. over C [0.001, 100] log, class_weight ∈ {none, balanced}	C=0.421, class_weight=None	213/5000 iter.	73	+0.003
Hist. Gradient Boosting	30 cand. over learning_rate [0.02, 0.3] log, max_leaf_nodes [8, 64], min_samples_leaf [10, 600], l2_regularization [0.0001, 10] log, class_weight ∈ {none, balanced}	class_weight=None, l2_regularization=0.961, learning_rate=0.0329, max_leaf_nodes=15, min_samples_leaf=30	198/2000 iter.	22	+0.037

Table 3: Cross-validated and held-out test performance. Precision/recall/F1 are for the positive class at threshold t, tuned on out-of-fold training predictions; AP = average precision (PR-AUC). CIs: 1000 test-set bootstrap resamples.

Model	CV AP (mean±sd)	Test AP [95% CI]	ROC-AUC	t	Prec.	Recall	F1	Macro-F1	Bal. Acc.
Prior baseline	–	0.238	0.500	–	–	0.000	0.000	0.433	0.500
Logistic Regression	0.709±0.013	0.700 [0.684, 0.717]	0.888	0.32	0.585	0.771	0.665	0.769	0.800
Hist. Gradient Boosting	0.721±0.016	0.707 [0.691, 0.724]	0.890	0.35	0.605	0.750	0.670	0.775	0.799

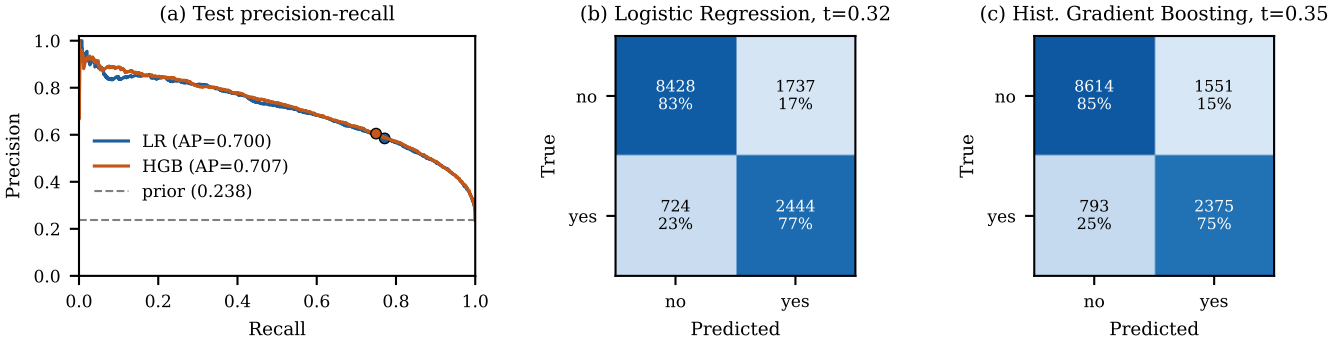


Figure 1: Held-out test set. (a) Precision–recall curves; dots mark the out-of-fold-tuned operating points, the dashed line the prior. (b, c) Confusion matrices at the tuned thresholds (counts and row-normalised rates).

balanced accuracy (CI -0.006 to +0.003) are statistically tied: HGB's edge lies in ranking, not at the chosen operating point, and is practically small (about one AP point). Threshold tuning matters more than the model: moving LR from 0.5 to 0.32 raises test F1 from 0.624 to 0.665 by trading precision (0.696→0.585) for recall (0.566→0.771); at the tuned thresholds LR misses 724 and HGB 793 of 3,168 test positives (Fig. 1), at the cost of more false alarms.

Table 4: Ablations: one decision reversed, tuned hyper-parameters kept (so Δ is conservative for the variant). Δ = change in CV average_precision on the same folds; sd = sd of paired fold differences ($|\Delta| <$ sd: no detectable effect); p: Nadeau–Bengio corrected t-test.

ID	Question	Δ average_precision	sd	p
A1_lr_keep_zeros	LR: keep co-occurring zeros as values (no NaN + indicator)	+0.0002	0.0003	0.274
A2_hgb_keep_zeros	HGB: keep co-occurring zeros as values (no NaN + indicator)	+0.0004	0.0008	0.454
A3_lr_no_clip	LR: no winsorising of heavy tails	-0.0006	0.0004	0.093
A4_lr_keep_composite	LR: keep the collinear composite_rank	0.0000	0.0000	0.788
A5_hgb_drop_composite	HGB: drop composite_rank	+0.0002	0.0015	0.821
A6_hgb_onehot	HGB: one-hot instead of native categorical splits	-0.0035	0.0009	0.005

5 Findings and discussion

(1) The signal appears largely additive. A linear model on one-hot features comes within about one AP point of boosting, so interactions and non-linear thresholds add little here (with the caveat that the two pipelines also differ in preprocessing). **(2) Few preprocessing decisions mattered.** Reverting native categorical splits to one-hot changed HGB AP by -0.0035 (A6, p=0.005); removing LR's winsorising changed AP by -0.0006 (A3: beyond fold noise but p=0.093, weak evidence); zero handling (A1, A2) and the `composite_rank` choice (A4, A5) changed AP by less than the fold-to-fold noise. The plan predicted that HGB would profit from `composite_rank`; the ablation does not support this, even though permutation importance ranks it second for HGB (0.054). Its information is shared with `aptitude_score` and `performance_score`: important is not the same as necessary. **(3) The models read the data differently.** LR leans on `stability_index` (importance 0.259), whereas HGB spreads importance over categoricals (`personal_interest`, `instrument`); `geological_era`, `load_ratio` and `index_weight` contribute nothing to HGB. **Limitations.** One train/test split; thresholds chosen on the folds that chose hyper-parameters (mildly optimistic); F1 is arbitrary without error costs; anonymised features rule out a domain check of the zero blocks. **Next steps:** LR with spline features, to separate non-linear main effects from interactions, and a cost-based threshold once costs are known. **Recommendation:** if transparency matters, LR gives up little; otherwise HGB, tuned to the operating point the application needs.

Reflection (not counted toward the two-page limit)

Human oversight. I stayed in the loop mainly by *designing the constraints* rather than by watching each step. Before any code existed I settled, in three question rounds with the agent, the decisions that define the skill: Variant 2; a hybrid design in which deterministic scripts compute every number and the LLM only decides and argues; exactly two contrasting models plus a prior baseline; CV-only model selection with a single use of the test set; AP and macro-F1 as primary metrics; and generalisation tests on two other datasets. I delegated both checkpoints (plan, report) to *fresh* reviewer sub-agents that saw only the files, never the author's reasoning, and required every point and its resolution to be logged (`runs/in6227/oversight_log.md`). Those reviews changed the run. The plan review added a missing ablation (zero handling for HGB) and showed that ablation deltas were measured against a selection-biased reference; the skill now reports the sd of paired fold differences and treats $|\Delta| < \text{sd}$ as "no effect". The report review caught a false sentence ("both models miss under a quarter of positives"; HGB misses 25.03%) and two over-claims. I also changed the skill itself when it failed: the page check passed while a figure had silently floated to page 3, so it now compiles the main report separately and reads the page count from LaTeX. *What I would change.* Delegated review is not human review. Next time I would approve the plan myself at checkpoint 1, because that is where the judgement calls are (e.g. how to read the zero blocks). I would also freeze the plan and ablation list before the first full run: while the scripts were being debugged, one smoke run printed test metrics. This is disclosed in the log, and no decision traces to it, but a clean protocol would not allow it to happen.

Critical evaluation. The decision I find most questionable is treating the co-occurring zeros as missing (0→NaN plus indicator). The skill argued that zeros shared by several continuous measures mark a structurally different group; that is plausible, but the evidence cuts both ways. (i) The ablations show no benefit for either model (Δ = +0.0002 and +0.0004, both within fold noise), so the transformation adds complexity for nothing measurable. (ii) The 265-row block is 100% negative. If those zeros are recorded *after* the outcome, the indicator is a leakage channel; if they mean "no activity", it is a legitimate rule. With anonymised features the skill cannot tell the two apart, yet it turned an unresolved semantic question into a confident preprocessing step. I would have kept the raw zeros (simpler, equally accurate) and escalated the block to a human as a data question. A second questionable choice was keeping `composite_rank` for HGB on the grounds that trees cannot form linear combinations. The ablation refuted this (Δ = +0.0002). I agree with how the report handled it: it states the refuted prediction instead of hiding it.

Trustworthiness. Three layers. (1) *By construction*: prose numbers are tokens resolved from `results.json`, `profile.json` and `plan.json`; tables are generated from `results.json`; unresolved tokens abort the render; every literal typed in the prose is listed in `numbers_audit.md`. (2) *Independent review*: the reviewer agent re-derived the HGB tuned-threshold precision, recall, F1 and balanced accuracy, the LR CV AP mean±sd, the baseline macro-F1 and every ablation sd, and found no mismatch. (3) *Manual check outside the pipeline*: with plain pandas on the raw CSV files I recomputed the labelled row counts (31,109 / 13,333), the positive share (0.2399) and the 265-row all-negative zero block. I confirmed that `composite_rank` $-(0.006\,$`aptitude_score`$\,+\,0.004\,$`performance_score`$)$ never exceeds 5×10^{-4} in absolute value, and that HGB's reported recall 0.7497 equals 2375/3168 from its confusion matrix. The one error found, the "under a quarter" sentence, was the only claim written as an inference in prose rather than as a token. That is the lesson: the token mechanism protects numbers, not the conclusions drawn from them, so conclusions still need a human or adversarial reader. Finally, generality was tested rather than asserted: the skill was run on two very different datasets (`examples/`: numeric binary without a test file; categorical 4-class). Those runs exposed two over-sensitive detectors (one flag per correlated pair; KS drift firing on a 114-row hold-out), which I fixed. All three reports were then regenerated with the same final code.